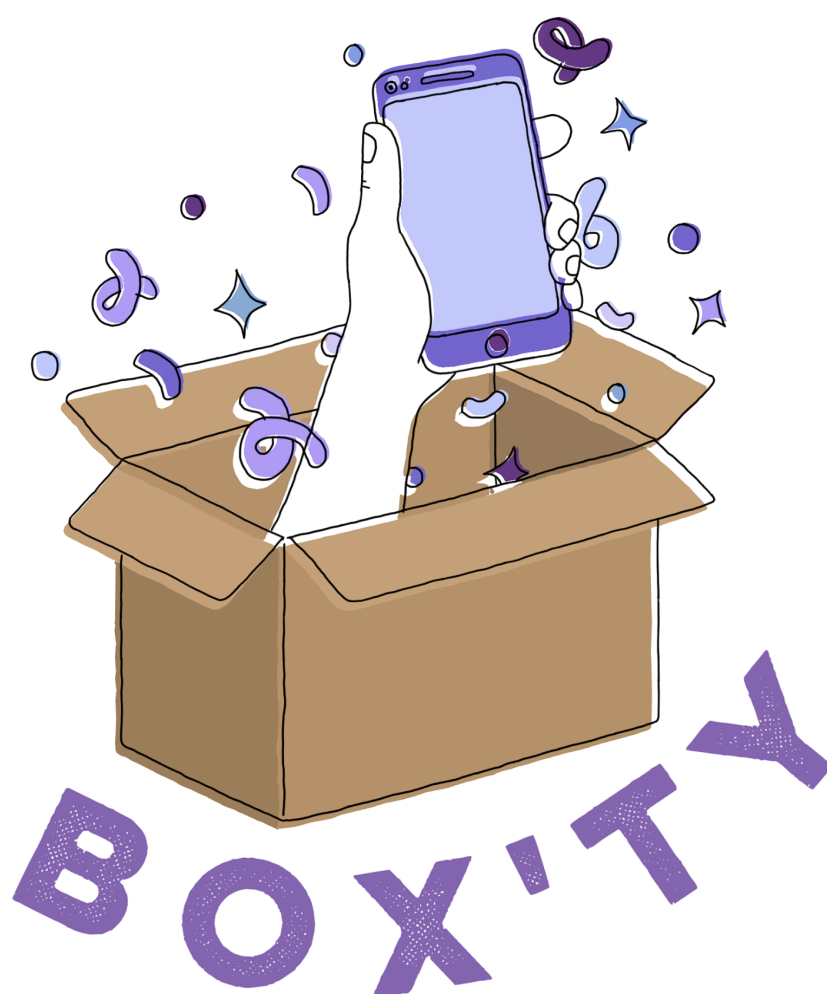


DOSSIER DE MAINTENANCE ET D'ADMINISTRATION **BOX'TY**

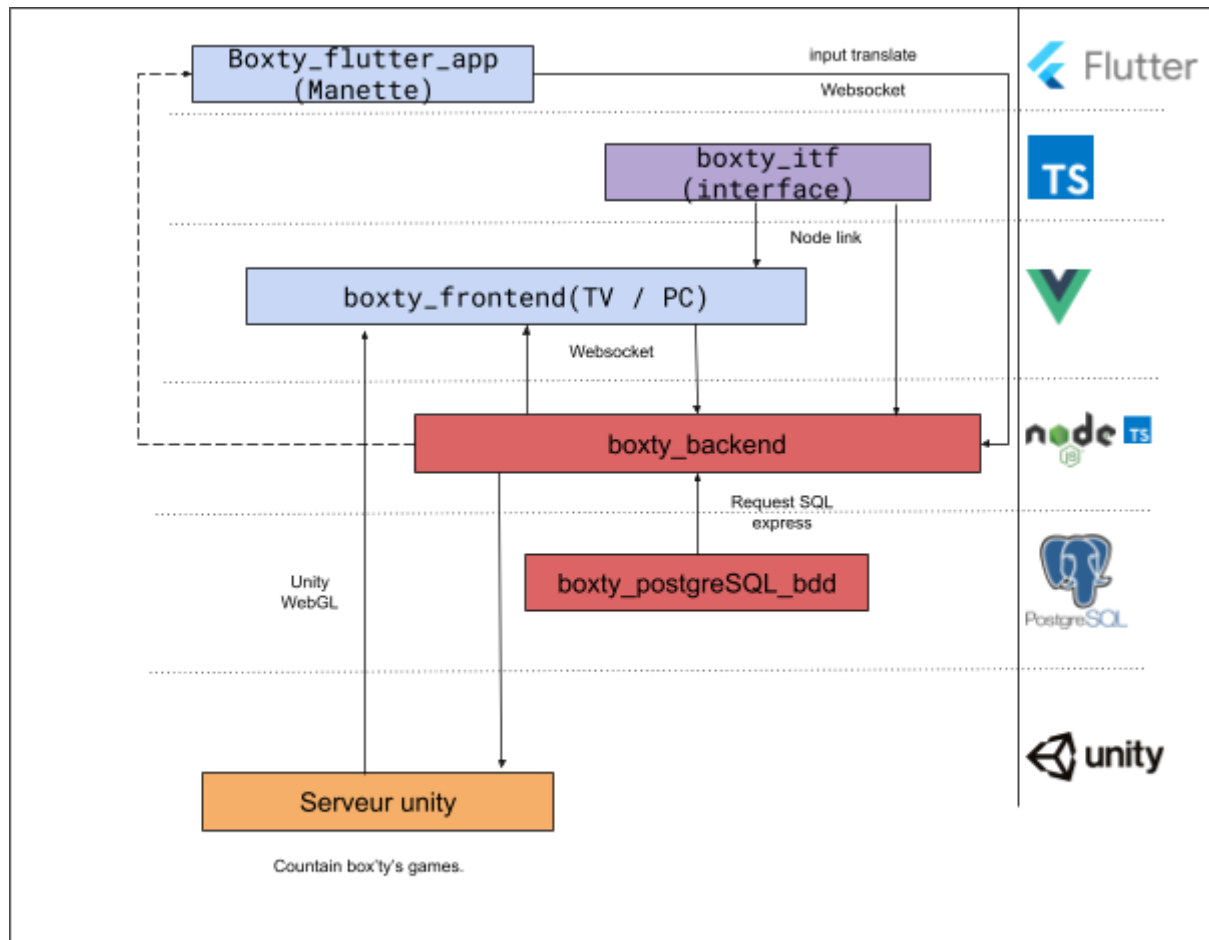


Antonin PULBY | Maxime PAJARD
Timothé TORRES | Kevin FAGOT | Maxime JOULLIN

SOMMAIRE

I / Architecture globale de l'application	3
A / Description de l'architecture	3
B / Les infrastructures mises en place	4
II / Installation et configuration	4
A / Demander l'accès au repo	4
B / Cloner le projet Git	5
C / Lancer le projet	5
1 - Docker	5
2 - En local sans Docker	6
3 - Lisez-moi	6
III / Note de Cadrage	7
A / Définition du projet	7
B / Objectif du projet	7
C / Périmètres	7
D / Contraintes	7
E / Acteurs	7
F / Ressources	7
G / Communication	8
H / Risques	8
IV / Outils de Développement	8
A / Environnement de développement	8
B / Git	8
C / Drive & Trello	9
D / Discord	10
V / Modules et Bibliothèques utilisés	10
VI / Stockage de données	10
A / Installation et mise en place de pgAdmin en local	10
B / Copier le back-up pour avoir la BDD en local.	12
VII / Déploiement Unity	12
VIII / Système pris en charge & périphériques mobiles	13
IX/ Rapport d'incident	13

I / Architecture globale de l'application



Everything is set up on an AWS server

A / Description de l'architecture

- Les joueurs sont sur une application Flutter pour se connecter et participer à un salon de jeu. L'un des joueurs est alors considéré comme hôte pour contrôler la mise en place, la configuration et le déroulement du jeu.
- Les inputs de Flutter sont émis via WebSocket
- Les WS sont reçues par un Back en TypeScript et Node.js pour traduction et renvoient vers le front

- Le front JS est en Vue.js, ce sera l'interface principale que les joueurs vont regarder.
- La sauvegarde des comptes utilisateurs est stockée sur une Base de données PostgreSQL
- Les requêtes sont faites en Express.js
- Les différents jeux sont développés sous Unity, ils sont intégrés via un Build WebGL au serveur, ils seront téléchargés sur le FrontEnd lors du lancement de la partie.
- Connexion à un écran de SmartTV / Ordinateur
- L'hôte de la partie peut faire la mise en place soit depuis son propre téléphone soit depuis l'ordinateur où sera joué le jeu.
- Chaque utilisateur peut choisir son nom / se connecter sur son téléphone, puis rejoindre avec un code partie affiché sur la TV / écran ses ami(e)s.

B / Les infrastructures mises en place

Pour la mise en place d'un environnement de travail rapide à l'utilisation pour nos développeurs nous avons conteneurisé notre application avec Docker.

<https://docs.docker.com/docker-for-windows/install/>



Notre infrastructure de production est hébergée sur un serveur Amazon Web Service (AWS), dans lequel nous plaçons le code fonctionnel, testé et vérifié, ce dernier est loué par l'équipe dans le cadre du développement de l'application.



Enfin, l'équipe possède le site boxty.fr hébergé sur OVH.com qui sera notre site de production une fois le logiciel en V1.



II / Installation et configuration

A / Demander l'accès au repo

L'équipe Box'ty dans son cadre de travail utilise l'outil de gestion pour le développement logiciel **GitHub** ainsi que l'outil de versioning **Git**.

Pour installer git :

<https://git-scm.com/book/fr/v2/D%C3%A9marrage-rapide-Installation-de-Git>

Pour avoir accès au repo du groupe Box'ty il faut faire la demande d'accès en tant que développeur à boxty.fr@gmail.com pour le repo de travail.

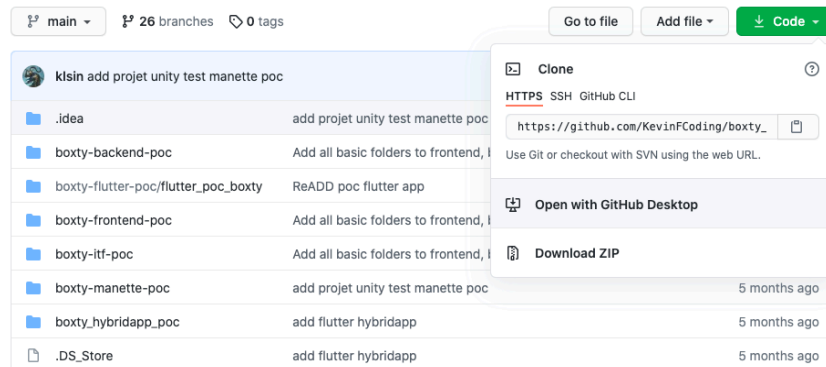
B / Cloner le projet Git

Rendez-vous sur le lien suivant:

https://github.com/KevinFCoding/boxty_poc

Cliquez sur Code, puis copiez l'url du repo.

Dans votre gestion git (que ce soit un GUI comme Git Kraken ou dans un terminal), faites un git clone dans le dossier que vous souhaitez de l'url copié :



C / Lancer le projet

1 - Docker

Une fois cloné, vous pouvez voir que le projet est un seul gigantesque repo contenant les 3 projets ainsi que le système d'interface ITF.

Vous avez la partie Application Mobile en Flutter : *boxty-flutter*



La partie FrontEnd en Vue JS : *boxty-frontend*



La partie BackEnd en NodeJS : *boxty-backend*



Le projet Box'ty est intégralement dockerisé, cependant vous pouvez travailler en local pour la lancer.

Pour lancer via Docker, ayez d'abord Docker d'installer sur votre outil de développement, ensuite placez-vous à la racine du projet et faites :

```
docker-compose build suivi d'un docker-compose up
```

Cela lancera entièrement l'application sur les conteneurs Docker disponible en <https://localhost> :

<https://localhost>

2 - En local sans Docker

Vous pouvez sinon lancer chaque application à la main si vous ne souhaitez pas utiliser Docker, pour cela :

Dans le FrontEnd, se rendre dans boxty-frontend, on commence par installer les node-modules à partir du package.json si ce n'est pas déjà fait :

```
npm install
```

Une fois les packages requis installés il n'y a plus qu'à lancer :

```
npm run serve
```

Pour le Backend, nous faisons la même chose, on se rend dans boxty-backend puis on fait :

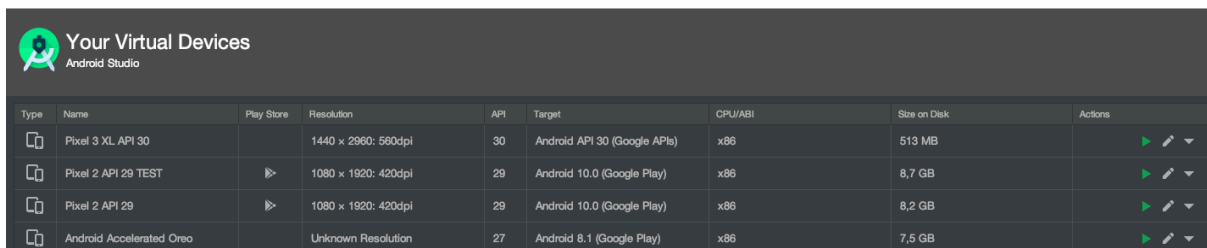
```
npm install && npm start
```

Le Backend étant en typescript, npm start utilise le script :

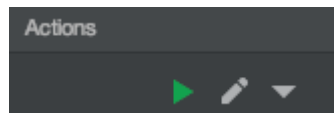
```
"start": "tsc && node dist/src/app.js",
```

Pour compiler le typescript en JS qui sera ensuite interprété par le navigateur.

Enfin pour flutter, il vous faut accès à un AVD manager (donc soit utiliser celui Android intégré dans IntelliJ ou Android Studio) ou un autre système d'émulateur de SmartPhone :



Type	Name	Play Store	Resolution	API	Target	CPU/ABI	Size on Disk	Actions
	Pixel 3 XL API 30		1440 x 2960: 560dpi	30	Android API 30 (Google APIs)	x86	513 MB	
	Pixel 2 API 29 TEST		1080 x 1920: 420dpi	29	Android 10.0 (Google Play)	x86	8,7 GB	
	Pixel 2 API 29		1080 x 1920: 420dpi	29	Android 10.0 (Google Play)	x86	8,2 GB	
	Android Accelerated Oreo		Unknown Resolution	27	Android 8.1 (Google Play)	x86	7,5 GB	



Lancer un appareil émulé :

Puis lancer dans la top bar de votre application Android ou IntelliJ le main.dart sur votre émulateur.



3 - Lisez-moi

Des read-me sont placés à la racine de tous les projets pour un récapitulatif pour savoir comment lancer chacun d'entre eux, ou alors comment les lancer tous d'un coup via Docker.

III / Note de Cadrage

A / Définition du projet

Box'ty est une plateforme de jeu en ligne disponible sur navigateur et permettant de jouer avec son téléphone en manette et avec ses amis à des jeux directement affichés et hébergés sur le navigateur.

B / Objectif du projet

Créer une plateforme de jeux très facile à utiliser et d'accès pour pouvoir jouer entre amis en soirée sans avoir à ramener de matériel supplémentaire que sa présence et son téléphone. Le projet Box'ty est hébergé sur un site Web qui fera en même temps vitrine pour l'équipe de développement ainsi que de suivi pour les jeux.

C / Périmètres

Le projet aura un site web vitrine avec la possibilité de s'y inscrire et de s'y connecter, d'y obtenir des informations au sujet de l'équipe, il y aura pour la V1 une simple inscription et gestion de compte.

L'application mobile / manette, se modifiera en fonction du jeu lancé par l'hôte de la partie et l'utilisateur pourra se connecter, néanmoins la manette ne servira pas à s'inscrire.

Le site sera donc principalement un site de service pour jouer à des jeux, les utilisateurs ne pourront pas créer de jeu à intégrer dans la plateforme, les jeux seront développés par l'équipe Box'ty.

D / Contraintes

Le projet étant développé dans le cadre d'une année scolaire, nous jonglons entre l'entreprise, les cours et le projet en lui-même, le facteur temps étant de rigueur, il nous est imposé de mettre en place un système de travail collaboratif efficace qui prend en compte les disponibilités de chacun.

E / Acteurs

Notre scrum master est Antonin Pulby, il travaille en tant que PO en même temps que développeur, Maxime Joullin, Maxime Pajard, Timothée Torres et Kevin Fagot sont tous les quatre développeurs sur le projet Box'ty.

La validation des livrables se fait suite à des Codes Reviews en interne pour valider l'envoi sur l'environnement de production.

Maxime Joullin est le lead développeur pour tout ce qui touche à Unity, Kevin Fagot s'occupe de l'architecture logicielle du projet, Maxime Pajard s'est occupé de la partie réseau et base de donnée tandis que Timothée Torres s'est occupé du développement frontEnd.

F / Ressources

Nous sommes une équipe forte de 5 développeurs encadrés par Robin Penea, Fondateur de Gryt, Ingénieur Software et Product Owner à la Coding Factory.

Nous avons l'aide d'une designeuse externe : Kenza Kouadria, qui nous aide pour les logos et le design.

Nous utilisons les bureaux de la Coding Factory pour nous rassembler et travailler en équipe, nous avons tous du matériel Windows et Mac pour le développement.

Nous avons un serveur AWS, le nom de domaine Boxty.fr, un Trello, un discord, un Google Drive dédié et un GitHub privé pour notre projet.

G / Communication

Nous faisons plusieurs audits "commerciaux", une fois tous les deux mois.

Nous travaillons en sprint de deux semaines, nous faisons un point tous les dimanches à 18h30 pour préparer la semaine de travail, jour durant lequel nous décidons d'un soir dans la semaine pour nous retrouver et travailler à distance.

Nous faisons un point daily à chaque fois que l'on travaille sur le projet.

H / Risques

Le projet étant complexe, nous avons déjà décidé de réduire le scope, néanmoins les contraintes pourraient occasionner un retard dans le développement, pour cela nous faisons un maximum de revues sur le Sprint pour réviser le backlog.

Latence dans les input de la manette, notre envie de créer des jeux et d'utiliser un téléphone en manette pourrait créer un certain "delay input" que l'on cherche à palier, réduire et vérifier.

IV / Outils de Développement

A / Environnement de développement

En général l'équipe Box'ty utilise IntelliJ pour pouvoir utiliser un seul IDE dans le cadre du développement de l'application, la possibilité d'y ajouter un émulateur nous permet de passer par ce dernier pour directement coder en Flutter, en Vue JS et en NodeJS.

Cependant, il n'y a pas de soucis pour nous à utiliser Visual Studio Code pairé à un Android Studio pour l'émulateur Flutter, nous demandons d'avoir un ESLINT d'installer pour NodeJS et VueJS pour permettre un code plus propre et clair.

Pour la base de données nous utilisons PostGres, nous avons pgAdmin d'installé pour la gestion de la base de donnée en local.

B / Git

L'équipe utilise donc le repo git de Box'ty pour développer l'application, cela permet à chacun d'avoir l'œil sur les avancées de chaque partie de l'application et de participer aux pull-requests et aux codes reviews des différents projets.

Le méta-repo ayant différents langages de programmation nous avons mis au point certaines règles pour faciliter la lecture des branches et des PR :

frontend/[nomFeatures] si vous travaillez sur la partie vues.js en FrontEnd pour le site.

backend/[nomFeatures] si vous travaillez sur la partie nodeJS en backend

itf/[nomFeatures] si vous travaillez sur la partie en TS pour les interfaces

nomDuJeu/[nomFeatures] en fonction du jeu UNITY sur lequel vous travaillez

manette/[nomFeatures] si vous travaillez sur la manette FLUTTER

Si vous travaillez sur quoi que ce soit sur l'application, prévenez l'équipe lors d'un DAILY.

Nous avons une branche de développement : "develop", chacun des merges vers cette branche est soumis à une pull-request qu'au moins deux autres développeurs de l'équipe devront valider avant de merger.

Si la branche n'est d'après Git par mergeable, c'est au développeur faisant la PR de corriger les conflits avant de prévenir l'équipe pour le merge.

C / Drive & Trello

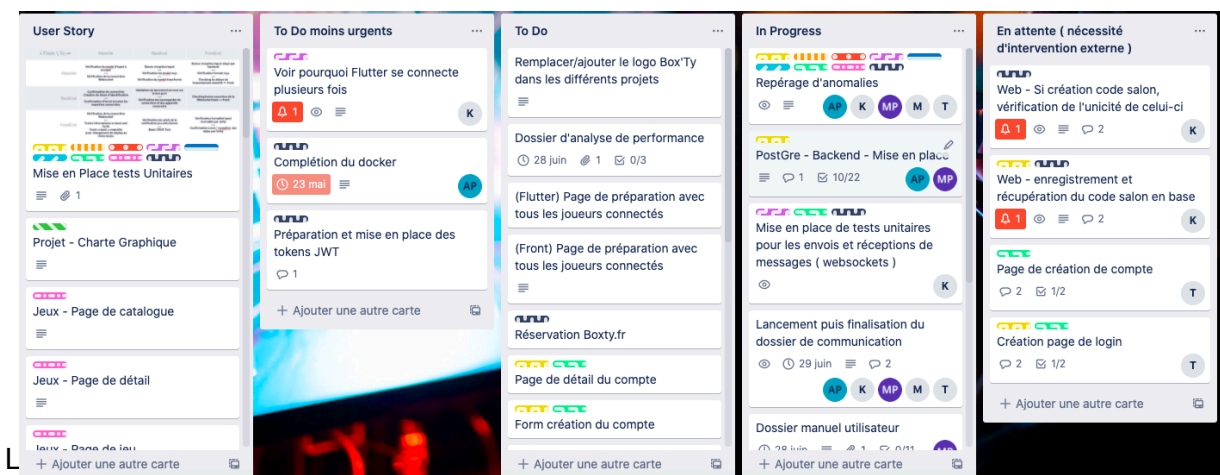
Avant d'avoir un Trello l'équipe possède un Drive dans lequel se trouve le backlog de l'équipe, les tâches ont été évaluées et priorisées en utilisant la méthode MoSCoW.

Priorisation :		Méthode MoSCoW
	M	Must Have
	S	Should Have
	C	Could Have
	V	Would Have

Nous avons donc un excel récapitulatif des différentes features et de leur état de développement, de plus c'est sur ce backlog que nous faisons nos sprint planning et que nous découpons le backlog lorsque les stories sont des epics.

	A	B	C	D	E	F	G
1	BACKLOG - Box'ty						
2	Sommaire						
3	Catalogue de jeux	Fiche détail du jeu	En cours	<- Liens vers les différentes features de Box'ty	Fait	Avancement des storys	
4		Stockage des jeux (serveur)	En cours				
5	Aspect compétitif	Tournoi	Pas commencé		Terminé dans le sprint en cours		
6		Scores	Pas commencé				
7	Application mobile manette	Gestion de l'hôte	Pas commencé		Dans le sprint en cours (Lien Trello)		
8		Interactivité manette / jeu	En cours				
9	Intégration de publicités		Pas commencé		À venir		
10	Site et informations	Partenariats	Pas commencé				
11		Accueil et informations diverses	Pas commencé				
12	Personnalisation et système économique	Création et personnalisation de parties	En cours				
13		Achats et paiements	Pas commencé				
14	Multi-plateforme & déploiement	Mobile hybride (IOS - Android)	En cours				
15		Architecture serveur et docker	En cours				
16	Traduction dans plusieurs langues		Pas commencé				
17	Gestion de comptes utilisateurs et BDD		En cours				
18	Création de marque et design		En cours				

Le trello recoupe les user stories du projet tiré du backlog, un code couleur est à disposition pour retrouver facilement de quel type de feature les todos découlent pour les développeurs.



différents crash-tests qui ont été opérés durant le développement de l'application.

Il nous permet non seulement de communiquer mais aussi de préciser sur quoi on travaille, de le partager et aussi de ne pas refaire les mêmes erreurs qu'avant, en gardant la trace et l'historique de nos avancées.

Le discord regroupe aussi les différentes dates de rendus, les différents rendez-vous programmés pour les audits et rendus d'avancement mais aussi fait office de rappel pour les différents développements sur les multiples projets.

V / Modules et Bibliothèques utilisés

Dès le début de l'application nous avons décidé de partir sur du TypeScript en langage de développement pour notre NodeJS et notre ITF.

Nous utilisons Vues DevTools pour nos informations de performances venant de notre FrontEnd et Flutter DevTools pour l'application mobile.

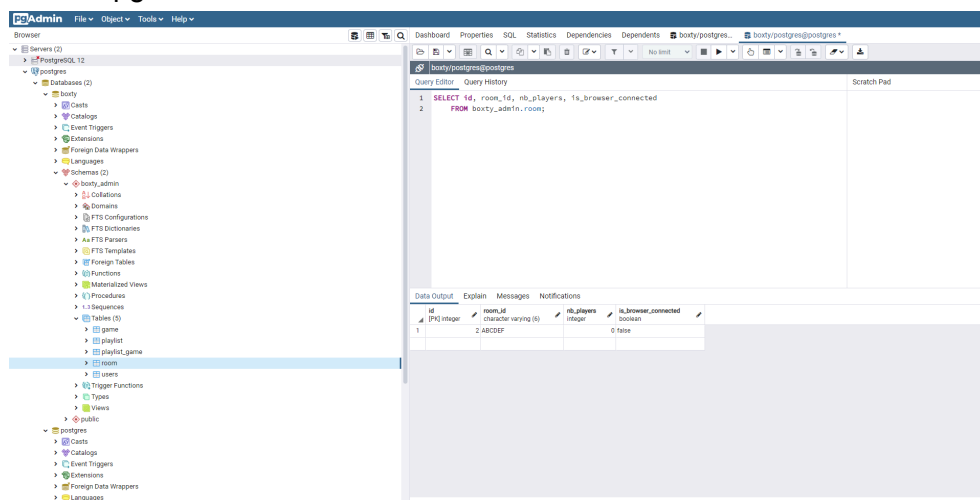
Pour le transfert des diverses données et informations entre les applications, nous utilisons express dans le BackEnd pour monitorer deux ports : le 8000 qui est le frontEnd et le 8001 ou se connecte les manettes.

Les envois de données sont faites comme précisés dans l'architecture via WebSocket, des models de données prédéfinis sont prévus pour éditer les données avant de les envoyer.

VI / Stockage de données

A / Installation et mise en place de pgAdmin en local

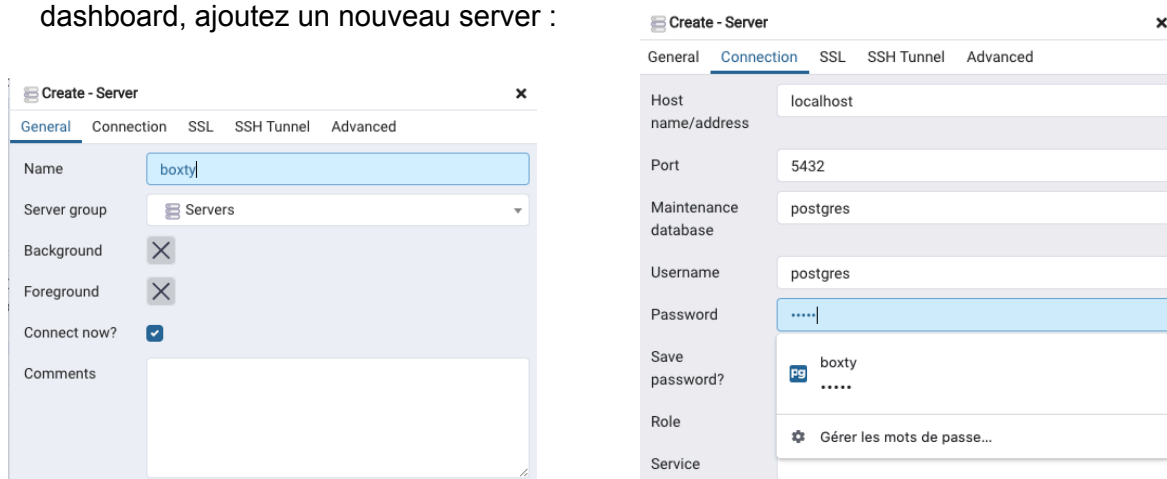
Comme précisé précédemment, Box'ty à une base de données en PostGres, cette dernière manageable via pgAdmin en local :



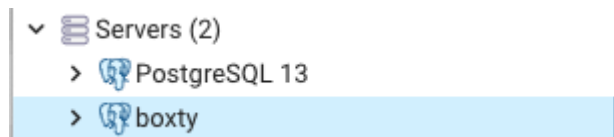
Pour y avoir accès, il faut donc installer la dernière version de pgAdmin :

<https://www.pgadmin.org/download/>

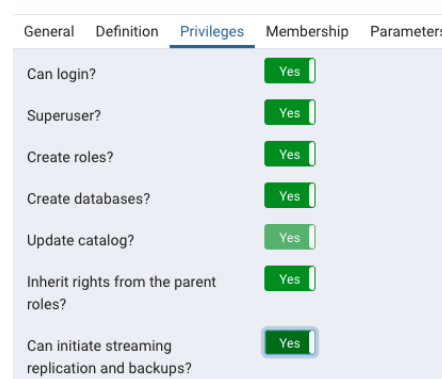
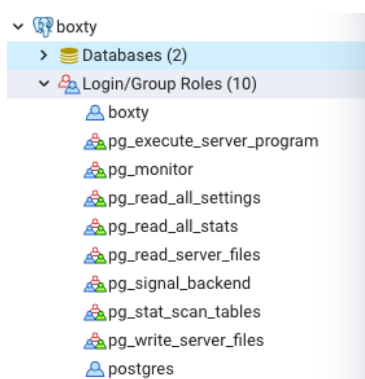
Une fois pgAdmin installer, dans le dashboard, ajoutez un nouveau server :



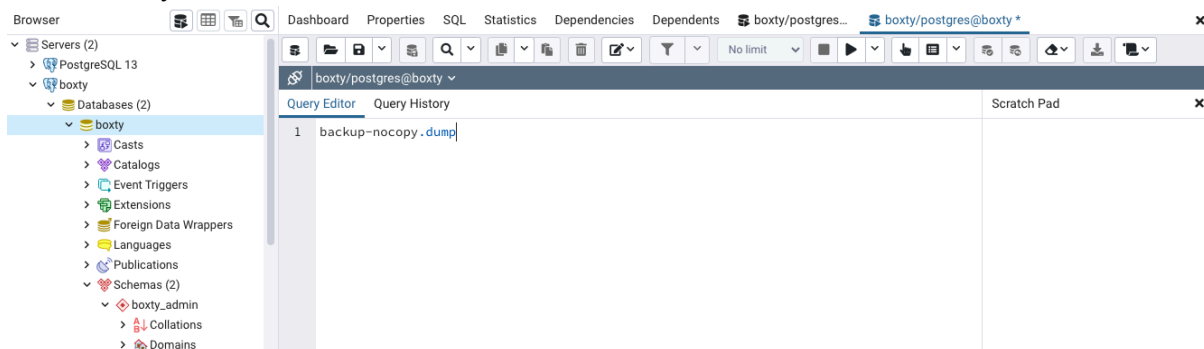
Lui donner en nom de server et password “boxty”, déclarer l’hôte en localhost.



Dans les logins/roles de Boxty, rajoutez un rôle d’administrateur “boxty” avec tout les privilèges.



Faire un clique droit sur le serveur, créer une nouvelle database appelé boxty avec pour owner “boxty”.



Aller dans le projet cloner, récupérer dans le dossier Docker le back-up-nocopy.dump

docker/postgres/backup-nocopy.dump

Recopier le back-up dans le query editor et lancer le code pour créer les tables, vous avez désormais la base de donnée en local, il vous suffit d’utiliser le .env à récupérer sur le discord pour travailler avec les variables locales, sinon utiliser les variables par défaut pour travailler avec Docker.

VII / Déploiement Unity

Notre projet étant aussi composé de différents jeux créés avec Unity, il est important de savoir extraire un build adapté à l'architecture du projet.

Ainsi, une fois le projet Unity terminer, il faudra créer un **“build”**

Il faudra se rendre dans le menu “Build Settings” (*Ctrl + shift + B*)

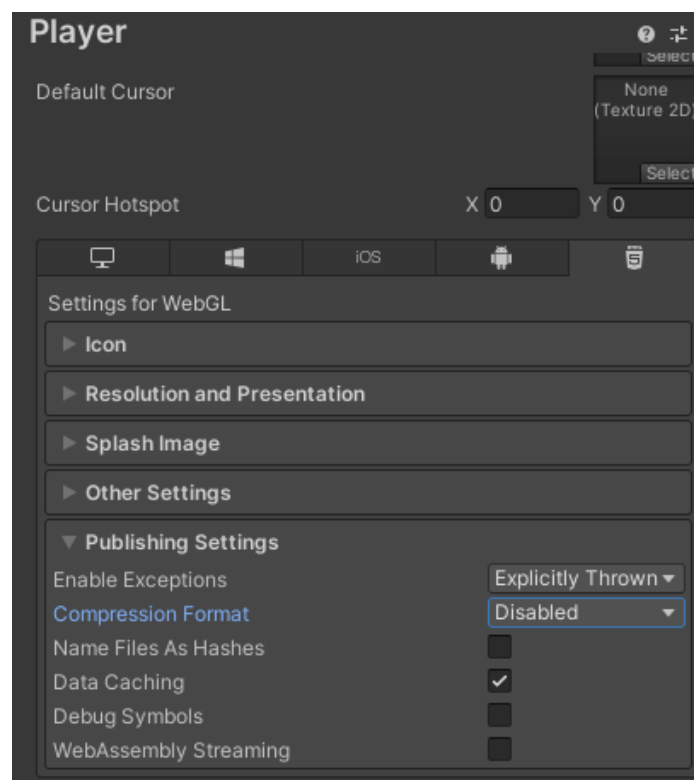
Dans ce menu, sélectionner un build WebGL :



Ensuite, se rendre dans les *Player Settings*



Puis dans l'onglet “*Publishing Settings*”, positionner l'option “*Compression Format*” sur *Disabled*



Une fois effectué, vous pouvez lancer le build, qui sera ensuite à positionner dans le dossier ***public*** du projet “***boxty-frontend***”

VIII / Système pris en charge & périphériques mobiles

Box'ty est hybride et utilise une technologie web bien supporté par les différents navigateurs, le projet est aujourd'hui compatible avec :



IX/ Rapport d'incident

Nous avons aussi mis en place un système de reporting d'incident sur un excel, nous avons aussi un compte sur FreshDesk, un outil gratuit de Ticketing pour le suivi de notre Dette Technique et de nos incidents :